Reinforcement learning for symbolic mathematics

Kevin, O'Keeffe [*1]

Abstract

We explore whether reinforcement learning (RL) can be useful for symbolic mathematics. Previous work showed contrastive learning can solve linear equations in one variable. We show that model-free PPO (Schulman et al., 2017) combined with a graph-based policy (TreeMLP), a dynamic per-state action space, and lightweight inductive biases (curriculum, success replay, constant relabeling) can solve nonlinear equations involving radicals, exponentials, and trigonometric functions—reaching or exceeding state-of-the-art accuracy on the CommonCore benchmark. We further consider *open* equations whose solution requires a generative change-of-variables, and train a separate substitution policy π_{cov} to handle them.

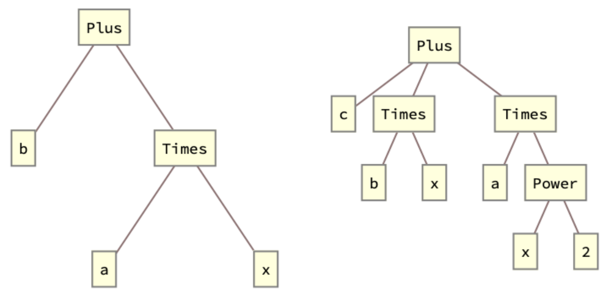

Figure 1. Expression trees for $ax + b$ and $ax^2 + bx + c$.

1. Introduction

Reinforcement learning (RL) has been applied to diverse fields (Ng et al., 2006; Degrave et al., 2022; Vinyals et al., 2019) but has not yet been widely adopted in symbolic mathematics, where agents perform tasks like solving algebraic equations or evaluating integrals analytically. Traditional approaches to symbolic mathematics rely on hand-engineered systems like Mathematica or Maple. An RL-based approach, where agents learn transformations autonomously, could reduce manual curation and find novel solution techniques.

Applying RL to symbolic math is difficult because the state space is combinatorially large, as each equation can branch into numerous sub-expressions. The action space is also large and dynamic: at every step, the agent can apply various algebraic manipulations to different sub-expressions, so the number and type of actions change with the equation's form.

We show that RL agents combined with a graph-based policy (TreeMLP), a dynamic per-state action space, and lightweight inductive biases (curriculum learning, success replay, constant relabeling) can overcome these challenges. The agents learn to solve a wide range of algebraic equations including those involving radicals, exponentials, and trigonometric functions. Although perhaps simple from a human perspective, solving such equations with RL is nontrivial and—to our knowledge—has not been done before. Symbolic equation solving is also a key building block for more advanced tasks like solving differential equations, the natural next step.

2. Problem Formulation

Our Markov Decision Process (MDP) formulation is

States: equations like $ax + b = 0$ or $cx + d = -x/b$. We vectorize these using preorder traversal over the equation's expression tree (Figure 1) and pad to a maximum length $L = 50$ (long enough for the equation sizes we consider). Operations and symbols are mapped to integers, e.g. $\{\text{add}: 1,\ \text{sub}: 2,\ \text{mul}: 3,\ \ldots,\ x: 5,\ a: 6,\ b: 7,\ \ldots\}$. We encode both the lhs and rhs of the equation in this way and concatenate. For example, $x + a \to [\text{add}, x, a] \to [1, 5, 6, \text{PAD}, \ldots]$.

Actions: represented as $(operation, term)$ pairs, such as

[*]Equal contribution [1]Starling Research Institute, Seattle, USA. Correspondence to: Kevin, O'Keeffe <kevin.p.okeeffe@gmail.com>.

Proceedings of the 42^{nd} International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025.

(sub, b) or (div, a). Formally,

$$O = \{\mathrm{add},\ \mathrm{sub},\ \mathrm{mul},\ \mathrm{div}\} \cup \{\mathrm{square},\ \mathrm{sqrt},\ \mathrm{exp},\ \mathrm{log},\ \mathrm{sin},\ \mathrm{cos},\ \mathrm{asin},\ \mathrm{acos}\}, \tag{1}$$

$$T = \mathrm{SubExpr}(\mathrm{lhs}) \cup \mathrm{SubExpr}(\mathrm{rhs}), \tag{2}$$

$$A = (O \times T) \cup \{(\mathrm{expand}, \mathrm{None}),\ (\mathrm{collect}, x),\ (\mathrm{mul}, -1)\}. \tag{3}$$

Notice the first set of operations take two arguments, the second one argument. For the terms, we choose the list of sub-expressions in the lhs and rhs expression trees. Example of sub-expressions are

$$ax + b = 0 \Rightarrow \{a, x, ax, b\} \tag{4}$$

$$\frac{ax+b}{cx+d} + e = 0 \Rightarrow \{a, x, ax, b, c, d, cx, cx+d, ax+b\} \tag{5}$$

This term set is expressive enough to solve rational equations and all other equation types we consider in this paper. It is also dynamic: the list of sub-expressions is derived from the equation/state and thus has variable length. Looping back to the operations, we also include $(\mathrm{mul}, -1)$ and

$$\begin{array}{ll} \text{expand:} & cx + d + e(ax+b) \to aex + be + cx + d \\ \text{collect } x\text{:} & ax + bx \to (a+b)x \end{array}$$

These allow the agent to perform key algebraic steps (Appendix). Finally, we index the action set A serially, cap it at size $|A| = 50$[1] and mask illegal actions (e.g., division by zero, see Appendix).

Rewards. Define the *complexity* C of an equation as the total number of nodes and edges in the expression tree [2]. The complexities of the equations in Figure 1 are $C(ax + b) = 5 + 4 = 9$ and $C(ax^2 + bx + c) = 10 + 9 = 19$. The reward is then

$$R(\text{action}) = C(\text{equation}) - C(\text{equation after action}) \tag{6}$$

The intuition here is to encourage the agent to take actions which simplify the equation.

State Transition Function. We wrote our code in Python and used SymPy to apply operations to the equations. At each step, we keep track of a lhs and rhs of an equation e.g. $lhs = (ax + b)$ and $rhs = 0$. We apply actions to both the lhs and rhs. For example, (sub, b) results in $(lhs, rhs) = (ax, -b)$, and then (div, a) results in $(lhs, rhs) = (x, -b/a)$. The terminal condition for the environment is when $lhs = x$ and the substitution of the rhs into the original equation simplifies to 0.

Limitations. Importantly, this MDP formulation only works on equations which are 'closed,' in the sense that solving them requires manipulating the terms already present in the equation/sub-expression list. By contrast, solving 'open' equations requires adding new, out-of-equation terms or clever substitutions. A classic example is the quadratic equation $ax^2 + bx + c$ which is solved by completing the square – adding $(b/2a)^2$ to each side. This is 'generative' reasoning, since the term $(b/2a)^2$ is *not* in the term set we have defined. Equations that require these more exotic actions are beyond the scope of the current work (and were also not studied in all previous works (Poesia et al., 2021; Dabelow & Ueda, 2024)).

2.1. Inductive biases

We use a number of inductive biases to improve learning.

TreeMLP. We use a custom *TreeMLP* architecture to exploit the expression-tree structure of our equations (nodes are tokens/operators; edges are parent→child). Each node ID is embedded and linearly projected to a hidden size, then we perform K rounds of message passing with a simple sum aggregator over incoming edges (no degree normalization). At round t, a node's representation is updated by applying a two-layer MLP with ReLU to the concatenation of its static embedding and the aggregated message, i.e., $h_i^{(t+1)} = \mathrm{MLP}\Big(\big[\mathrm{emb}_i \,\|\, \sum_{j\to i} h_j^{(t)}\big]\Big)$. Padding-aware masks zero out invalid nodes and edges at every stage. After K iterations, we obtain a fixed-size graph feature by pooling (mean or max) over valid node states, which is fed to the policy/value heads. Compared to standard GCN/SAGE extractors, TreeMLP avoids degree normalization and learned edge transforms, yielding a lightweight, strictly local update that aligns well with symbolic expression trees and trains stably with PPO.

We give more details on TreeMLP in the Appendix. We also show this TreeMLP outperforms other popular graph based models, like GNNs and GraphSage (Figure 5).

Curriculum learning. We consider multi-equation environments, where the agent solves a single equation chosen randomly from a larger set during each episode. Equations are selected via *inverse sampling*, where the probability of choosing an equation is inversely proportional to the number of times it has been solved before. This curriculum ensures the agent spends more time trying to solve equations it hasn't solved before.

Success replay buffer. A useful property of our environment is that solution traces are *exact*: once a sequence of

[1]We explored other values like $|A| \in \{40, 70, 100\}$ and found no measurable change in performance.

[2]we judged number of nodes+edges correlates better with algebraic complexity than number of nodes alone.

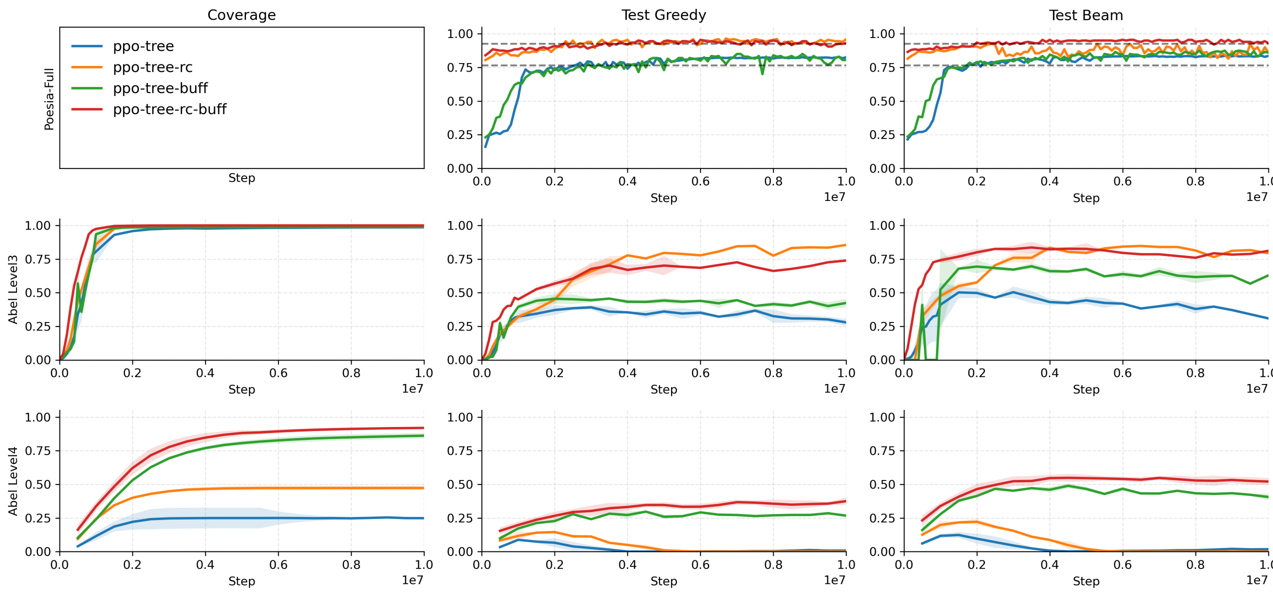

Figure 2. Learning curves for the closed environments. Mean of three random seeds, shading from min to max.

operations solves an instance (e.g., $ax + b = 0$), replaying the same (op, term) actions deterministically reproduces the solution. We exploit this by enabling a light-weight *success replay* path in the env, where the solution traces $(s_t, a_t)_t$ are stored to a memory buffer and periodically mixed into the on-policy rollouts.

Relabel constants. To improve parameter sharing across structurally identical equations that differ only by symbol names, we introduce a *relabel-constants* action. When invoked, it applies a consistent partial renaming (e.g., $\{d \mapsto a,\ e \mapsto b,\ \dots\}$) to the current main equation and the working state, mapping arbitrary coefficients onto a compact canonical alphabet $\{a, b, c\}$. The environment stores the induced substitution map on a stack (`map_constants_history`) and uses the mapped equation for all subsequent steps. Upon solving, we *unwind* the stack in reverse, substituting back the original symbols so the final answer is expressed in the user's variables. This reduces input vocabulary entropy, prevents spurious overfitting to symbol IDs, and makes action terms like "subtract b" reusable across many instances.

Curiosity based rewards. In previous work (Dabelow & Ueda, 2024), curiosity-based exploration bonuses (e.g. ICM, RND, NGU) were shown to enhance learning on a regular MLP policy. In preliminary experiments we found these bonuses did not improve, and in several cases degraded, performance when stacked on top of the TreeMLP; we therefore do *not* use them in our headline results (details deferred to a future revision).

Table 1. Greedy test accuracy for each agent across dataset. Single-seed results except where noted; ConPoLe / ConPoLe-local numbers from (Poesia et al., 2021). "–" denotes a cell awaiting a multi-seed re-run.

Algorithm	small	large	poesia
ConPoLe-local	n/a	n/a	0.765
ConPoLe	–	–	0.925
ppo	0.04	0.02	–
ppo-tree	–	–	0.815
ppo-tree-rc	0.90	–	0.96
ppo-tree-rc-buf	0.80	0.44	–

3. Results: Closed equations

Algorithms. We use the standardized implementation of ppo from stable-baselines 3. We experimented with A2C and DQN but found worse performance. Hyperparameters, chosen via tuning, are reported in the Appendix. We trained for 10^7 steps and evaluated every 5×10^5. We used three random seeds.

Datasets. We tasked the agent with solving a random equation during each episode. We generate train/test sets by starting with x and applying actions recursively. Recall actions are $(operation, term)$ pairs. The operations are as before, but we restrict terms to (a, b, c) (we excluded d, e since we wanted to favor long, rather than wide equations). Applying a single action generated equations like $ax, \log(x), x+b$, applying 2 actions equations like $ax+b, \log(x)+d, (x+b)/c$ and so on. We threw out any equations that SymPy couldn't solve, and considered multi-root equations solved when just one root was found (e.g. $\sin(x) = 0$ is solved if the agent found $x = 0$ rather than $x = 2n\pi$). We created a small dataset which included all equations of $depth < 4$ (size 3874) and sub-sampled to a $10^3/10^2$ train/test split, and a large dataset of all equations of $depth < 5$ (size 15625) sub-sampled to a $10^4/10^3$ train/test split. Finally, we also use the CommonCore datasets used in previous works.

Results. Figure 2 shows the learning curve for each algorithm on all three datasets. We plot three performance metrics: (i) coverage, the fraction of train equations solved at least once, (ii) test accuracy following the greedy policy, (iii) and test accuracy following beam search of width 5. The top row shows the CommonCore datasets (with coverage excluded since the train set is random and unbounded). The dashed black lines show the results for ConPoLe-local and ConPoLe from (Poesia et al., 2021); these are the SOTA for symbolic equation solving. Notice our vanilla ppo-tree beats ConPoLe-local, while our inductive biases (ppo-tree-rc) beats ConPoLe. Table 1 reports the exact numbers.

The bottom two rows show the results on our small and large datasets. For the small dataset, all four methods achieve near-perfect coverage on the training set, but only PPO-TREE-RC and PPO-TREE-RC-BUF achieve high test accuracy under greedy decoding (≥ 0.80). For the large dataset, only PPO-TREE-RC-BUF attains both broad coverage and a usable test accuracy ($\text{test}_{\text{beam}} \approx 0.66$); vanilla PPO-TREE and PPO-TREE-RC collapse to near-zero greedy accuracy on the large set, suggesting the success-replay buffer is the critical ingredient at scale.

The takeaway is that our inductive biases help in each case, and are comparable to the SOTA.

Analysis. How does the agent learn to solve the equation? We show two solution traces below. Notice the policy is not always optimal: the second example takes a 3-step detour (steps 5–7: expand, collect, truediv) where a single multiply-by--1 would have sufficed, as in the first example's step 5.

1. Equation: $cx + d + e(ax + b) = 0$

```
Step 1: cx + d + e(ax + b) = 0 | expand, None
Step 2: aex + be + cx + d = 0 | collect, x
Step 3: be + d + x*(ae + c) = 0 | subtract, x*(ae + c)
Step 4: -x(ae + c) = be + d | truediv, ae + c
Step 5: -x = (be + d)/(ae + c) | multiply, -1
Solved: x = -(be + d)/(a*e + c)
```

2. Equation: $e + (ax + b)/(cx + d) = 0$

```
Step 1: e + (ax + b)/(cx + d) = 0 | truediv, 1/(cx + d)
Step 2: (e + (ax + b)/(cx + d))(cx + d) = 0 | expand, None
Step 3: ax + b + cex + de = 0 | collect, x
Step 4: b + de + x*(a + ce) = 0 | subtract, x(a
Step 5: -x(a + ce) = b + de | expand, None
Step 6: -ax - cex = b + de | collect, x
Step 7: x*(-a - ce) = b + de | truediv, -a - ce
Solved: x = (b + de)/(-a - c*e)
```

Per-type accuracy. The "small" dataset contains a heterogeneous mix of equation types. Bucketing the test set by structural class and re-evaluating the trained PPO-TREE-RC-BUF agent reveals a clear pattern (Table 2): linear and radical equations are nearly fully solved (≥ 0.88), polynomial equations of degree 2–4 sit in the 0.75–0.82 range, while transcendental forms—particularly trigonometric and logarithmic—are the hardest. The trigonometric bucket (113 equations, 72% accuracy) accounts for most of the remaining failures.

Table 2. Greedy test accuracy of PPO-TREE-RC-BUF on the small dataset, broken down by equation class.

Type	n	solved	acc
radical (e.g. $\sqrt{x}$)	47	43	0.915
linear (poly-deg1)	110	97	0.882
quadratic (poly-deg2)	34	28	0.824
log	77	58	0.753
poly-deg4	4	3	0.750
trig	113	81	0.717
overall	387	312	0.806

Learned representations. As a qualitative probe of what the TreeMLP encodes, we extract the pooled graph embedding for each test equation and project it to two dimensions with t-SNE (Figure 3). Points are colored by structural type. Equations form well-separated clusters by family—linear, radical, trig, and log each occupy distinct regions—suggesting the representation captures structural identity rather than surface tokens.

Failure modes. Manual inspection of the failed cases (75 of 387) shows three recurring patterns: (i) trig equations that require an arccos/arcsin step the agent never selects, (ii) log equations whose simplification creates a transcendental that breaks the closed-action set, and (iii) deep polynomials where the agent exceeds the per-episode action budget despite being on a productive path.

Scaling to the large dataset. The same per-type evaluation on the "large" dataset (1000 test equations of depth < 5) reveals a uniform drop in every category: linear $0.88 \to 0.60$, quadratic $0.82 \to 0.40$, log $0.75 \to 0.27$, trig $0.72 \to 0.36$, radical $0.92 \to 0.52$. The drop is largest for deep polynomials (poly-deg4: $0.75 \to 0.08$). Going from depth < 4 to depth < 5 approximately quadruples the test-set size and adds substantially deeper nesting, so the harder cases are genuinely harder, not just more numerous.

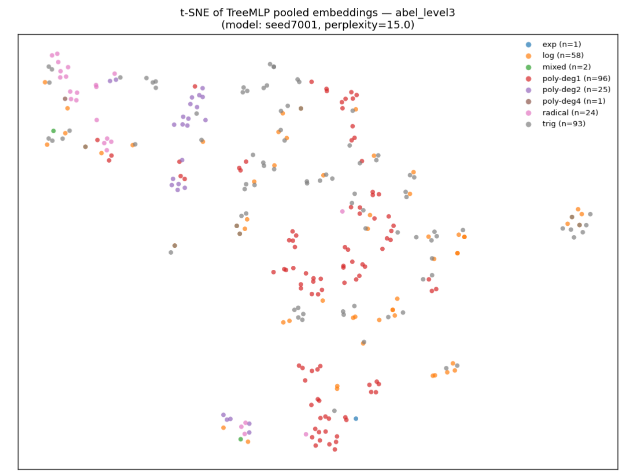

Figure 3. t-SNE projection of TreeMLP pooled embeddings for 300 small-dataset equations, color-coded by structural type. Clusters correspond to equation family rather than to specific coefficient assignments.

Generalization to CommonCore. On the CommonCore (Poesia) benchmark, the same PPO-TREE-RC agent reaches 0.955 greedy test accuracy. The breakdown is sharper: linear equations are solved at 0.98 and the small subset of rational equations at 1.00. The remaining failures are essentially edge-case templates that lack a free x in the polynomial decomposition (*degenerate* instances). Accounting for those, accuracy on the solvable subset is ≥ 0.98.

4. Results: Open equations

Now we consider the harder class of *open equations*. We train a separate change–of–variables policy π_{cov} which, given an input equation, proposes a substitution $x \mapsto \phi(x)$ that simplifies the instance into a canonical, easier-to-solve form. We consider four basic templates

$$\begin{aligned}
&ax^2 + bx + c = 0, && x \to x - b/(2a) \quad (7)\\
&ax^3 + bx^2 + cx + d = 0, && x \to x - b/(3a) \quad (8)\\
&ax^4 + bx^3 + cx^2 + dx + e = 0, && x \to x - b/(4a) \quad (9)\\
&ae^{kx} + be^{-kx} + c = 0, && x \to \tfrac{1}{k}\log x \quad (10)
\end{aligned}$$

The rhs of each shows the solving CoV. Rather than use a generative model (GAN/VAE) to propose ϕ, we frame π_{cov} as an RL problem.

MDP. **States** are the encoded main equation together with the current step depth d and an action history (last H action indices). The encoded equation does not change during an episode; only d and the history evolve as the agent builds ϕ. **Actions** are pairs (op, τ) with op $\in \{+, -, \times, \div\}$ and τ drawn from a small term bank $\{a, b, c, d, e, 2, 3, 4\}$, plus a special STOP action; the first action fixes a *base op*, and subsequent actions build an inner substitution u compositionally. **Transitions** are deterministic: at termination (STOP or $d = d_{\max}$), $\phi(x) := \text{base\_op}(x, u)$, and the env substitutes $x \mapsto \phi(x)$ in the main equation. **Reward** is the complexity-delta $C(\text{main}) - C(\text{main after } \phi)$ with a small per-step penalty; large positive bonus when ϕ reduces complexity (i.e., depresses the equation to a closed-equation-solvable form).

We train a CoV agent using PPO with TreeMLP. Figure 4 shows the results.

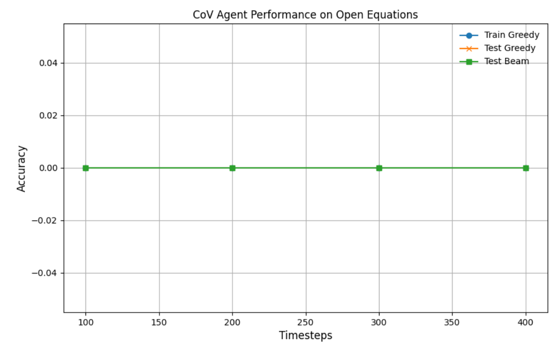

Figure 4. Performance of the CoV agent on open equations. The agent learns to propose correct substitutions to simplify the equations.

Acknowledgements

Impact Statement

This paper presents work whose goal is to advance the field of machine learning, specifically the application of reinforcement learning to symbolic mathematics. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Dabelow, L. and Ueda, M. Symbolic equation solving via reinforcement learning. *arXiv preprint arXiv:2404.17499*, 2024.

Degrave, J., Felber, F., Buchli, J., Neunert, M., Tracey, B., Carpanese, F., Ewalds, T., Hafner, R., Abdolmaleki, A., de Las Casas, D., et al. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897):414–419, 2022.

Langley, P. Crafting papers on machine learning. In *Proceedings of the 17th International Conference on Ma-*

chine Learning (ICML 2000), pp. 1207–1216, Stanford, CA, 2000.

Ng, A. Y., Coates, A., Diel, M., Ganapathi, V., Schulte, J., Tse, B., Berger, E., and Liang, E. Autonomous inverted helicopter flight via reinforcement learning. In *Experimental Robotics IX*, pp. 363–372. Springer, 2006.

Poesia, G., Dong, W., and Goodman, N. Contrastive reinforcement learning of symbolic reasoning domains. In *Advances in Neural Information Processing Systems*, volume 34, pp. 17729–17741, 2021.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575 (7782):350–354, 2019.

.1. Macroactions

Here we justify why the three macroactions $(\text{expand}, \text{None})$, $(\text{collect}, x)$, $(\text{mul}, -1)$ discussed in the main text are *required* to solve the equations we consider, not mere conveniences for the RL agent. Consider the action set *without* these macroactions:

$$\begin{aligned} O &= \{\text{add, sub, mul, div}\} \\ &\quad \cup \{\text{square, sqrt, exp, log, sin, cos, asin, acos}\}, && (11) \\ T &= \text{SubExpr(lhs)} \cup \text{SubExpr(rhs)}, && (12) \\ A &= (O \times T). && (13) \end{aligned}$$

Now consider the equation $dx + c(ax + b) = e$. The agent must distribute c into the bracketed term $ax + b$, which the action set above cannot do: expand is required. Once expanded to $dx + cax + cb = e$, one needs to factor the x common to the first two terms—this is what $(\text{collect}, x)$ provides. Finally, consider $-x = b$: since -1 is not in our term set, $(\text{mul}, -1)$ is needed to isolate x. (Alternatively, -1 could be added to the term set, but we chose to keep the term set purely symbolic.)

.2. Dataset generation: Rational Equation

To supplement the recursive datasets, we constructed rational equations of the form

$$\frac{ax + b}{cx + a} + b = 0,$$

where a, b, c are (non-zero) symbolic coefficients. We excluded degenerate cases such as denominators that vanish identically or cancel trivially with numerators.

Examples of retained equations include:

$$\begin{aligned} \frac{x + b}{cx + a} = 0 \quad &\Rightarrow \quad x = -b, \\ \frac{ax + b}{cx + a} + b = 0 \quad &\Rightarrow \quad x = \frac{-b - ab}{a + cb}, \\ \frac{ax + b}{cx + a} - c = 0 \quad &\Rightarrow \quad x = \frac{-b + ac}{a - c^2}. \end{aligned}$$

These rational forms were merged into both the small and large datasets, with their proportion limited to preserve diversity across other functional forms (logarithmic, trigonometric, exponential, etc.).

.3. Illegal actions

The main issue was to avoid illegal division by zero. This is not as simple as precluding $(\text{div}, 0)$ from the action set. We also had to prohibit *hidden* division by zero—for example, dividing by $(x + a)$ or $(x + b)$ when the equation has the form $(x + a)(x + b) = 0$, since one of the factors will vanish at the solution. To handle this, we check whether the current equation factors as $P(x)Q(x)\cdots = 0$ with $P, Q, \ldots$ polynomial in x, and remove $(\text{div}, P), (\text{div}, Q), \ldots$ from the action set for that state.

.4. Hyperparameters

We used Stable Baselines 3's default hyperparameters for A2C and PPO (Table 3).

Algorithm	Hyperparameters
PPO	$learning\_rate = 0.0003$, $n\_steps = 2048$, $batch\_size = 64$, $n\_epochs = 10$, $gamma = 0.99$, $gae\_lambda = 0.95$, $clip\_range = 0.2$, $ent\_coef = 0.0$, $vf\_coef = 0.5$, $max\_grad\_norm = 0.5$, $normalize\_advantage = True$, $device =' auto'$
PPO-tree	$learning\_rate = 0.0007$, $n\_steps = 5$, $gamma = 0.99$

Table 3. Default hyperparameters for A2C and PPO from Stable Baselines 3.

.5. Solution Traces

Below are the solution traces for each equation in the fixed equation environment. Notice the solution trace is not always optimal. For instance, $ax + b = 0$ could be solved with $(sub, b), (div, a)$, but instead the agent selects (sub, ax) and then $(mul, -1)$. "Truediv" is the sympy notation for divide.

1. Equation: $a + x = 0$

   ```
   Step 1: a + x = 0 | subtract, a
   Solved: x = -a
   ```

2. Equation: $ax = 0$

   ```
   Step 1: a*x = 0 | div, a
   Solved: x = 0
   ```

3. Equation: $ax + b = 0$

   ```
   Step 1: ax + b = 0 | subtract, ax
   Step 2: -ax = b | multiply, -1
   Step 3: ax = -b | div, a
   Solved: x = -b/a
   ```

4. Equation: $a/x + b = 0$

   ```
   Step 1: a/x + b = 0 | subtract, b
   Step 2: a/x = -b | truediv, 1/x
   Step 3: -b*x = a | truediv, b
   Step 4: -x = a/b | multiply, -1
   Solved: x = -a/b
   ```

5. Equation: $c(ax + b) + d = 0$

   ```
   Step 1: c*(ax + b) + d = 0 | expand, None
   Step 2: acx + bc + d = 0 | subtract, acx
   Step 3: -acx = bc + d | multiply, -1
   Step 4: acx = -bc - d | truediv, c
   Step 5: ax = (-bc - d)/c | truediv, a
   Solved: x = (-bc - d)/(ac)
   ```

6. Equation: $c + d/(ax + b) = 0$

   ```
   Step 1: c + d/(ax + b) = 0 | subtract, c
   Step 2: d/(ax + b) = -c | multiply, (ax + b)
   Step 3: d = -c (ax + b) | expand, None
   Step 4: d = -c ax - c b | multiply, -1
   Step 5: -d = c ax + c b | subtract, c b
   Step 6: -d - c b = c ax | truediv, c
   Step 7: (-d - c b)/c = ax | truediv, a
   Step 8: ((-d - c b)/c)/a = x | truediv, 1/a
   Solved: x = (-d - c b)/(c a)
   ```

7. Equation: $cx + d + e(ax + b) = 0$

Algorithm 1 TreeMLP message passing and pooling

Require: node IDs $\mathbf{X}$, edges (src, dst), masks $m_{\text{node}}, m_{\text{edge}}$, steps K

```
1: emb ← Proj(Embed(X))                          ▷ shape [B, N, H]
2: h ← emb
3: for k = 1 to K do
4:     ℰ ← {(i→j) | m_edge(i→j) = 1}
5:     agg[j] ← Σ_{(i→j)∈ℰ} h[i]                 ▷ scatter-sum
6:     h ← MLP([emb, agg])
7:     h ← h ⊙ m_node                            ▷ mask padded nodes
8: end for
9: return pool(h, m_node)                        ▷ mean or max
```

```
Step 1: cx + d + e(ax + b) = 0 | expand, None
Step 2: aex + be + cx + d = 0 | collect, x
Step 3: be + d + x*(ae + c) = 0 | subtract, x(ae + c)
Step 4: -x(ae + c) = be + d | truediv, ae + c
Step 5: -x = (be + d)/(ae + c) | multiply, -1
Solved: x = -(be + d)/(a*e + c)
```

8. Equation: $e + (ax + b)/(cx + d) = 0$

```
Step 1: e + (ax + b)/(cx + d) = 0 | truediv, 1/(cx + d)
Step 2: (e + (ax + b)/(cx + d))(cx + d) = 0 | expand, None
Step 3: ax + b + cex + de = 0 | collect, x
Step 4: b + de + x*(a + ce) = 0 | subtract, x(a + ce)
Step 5: -x(a + ce) = b + de | expand, None
Step 6: -ax - cex = b + de | collect, x
Step 7: x*(-a - ce) = b + de | truediv, -a - ce
Solved: x = (b + de)/(-a - c*e)
```

.6. TreeMLP Details

Architecture. TreeMLP embeds node IDs, projects to a hidden size, and performs K message-passing stages over the (directed) expression tree. At each stage it sums incoming neighbor states, concatenates the result with the fixed input embedding, and applies a two-layer MLP with ReLU. A masked global pool (mean or max) produces the graph embedding exposed to the policy/value heads.

Listing 1. TreeMLP forward pass (minimal).

```python
def forward(self, obs: dict) -> torch.Tensor:
    node_ids  = self._to_device(obs["node_features"], self.device)
    edge_index= self._to_device(obs["edge_index"], self.device)
    node_mask = self._to_device(obs["node_mask"], self.device)
    edge_mask = self._to_device(obs["edge_mask"], self.device)

    node_idx = self._id_to_idx(node_ids) if node_ids.dtype != torch.long else node_ids
    B, N = node_idx.shape
    _, _, E = edge_index.shape

    emb = self.embed(node_idx)                    # [B,N,embed_dim]
    emb = self.embed_proj(emb)                    # [B,N,H]
    h = emb.clone()

    src = edge_index[:, 0, :].long()            # [B,E]
    dst = edge_index[:, 1, :].long()            # [B,E]
    batch_idx = torch.arange(B, device=self.device).unsqueeze(1).expand(B, E)
```

```python
for _ in range(self.K):
    src_flat = src.reshape(-1)
    dst_flat = dst.reshape(-1)
    b_flat   = batch_idx.reshape(-1)
    m_flat   = edge_mask.reshape(-1).bool()

    src_flat, dst_flat, b_flat = src_flat[m_flat], dst_flat[m_flat], b_flat[m_flat]
    src_idx = b_flat * N + src_flat
    dst_idx = b_flat * N + dst_flat

    h_flat   = h.reshape(B * N, self.hidden_dim)
    messages = h_flat[src_idx]                          # [M,H]

    agg_flat = torch.zeros_like(h_flat)                 # [B*N,H]
    agg_flat.index_add_(0, dst_idx, messages)           # sum incoming
    agg = agg_flat.reshape(B, N, self.hidden_dim)       # [B,N,H]

    h = self.node_mlp(torch.cat([emb, agg], dim=-1))    # [B,N,H]
    h = h * node_mask.unsqueeze(-1).float()

if self.pooling == "mean":
    denom = node_mask.sum(dim=1, keepdim=True).clamp(min=1).float()
    g = h.sum(dim=1) / denom
else:
    if self._minus_inf is None or self._dtype_cache != h.dtype:
        self._minus_inf = torch.finfo(h.dtype).min
        self._dtype_cache = h.dtype
    g = h.masked_fill(~node_mask.unsqueeze(-1), self._minus_inf).max(dim=1).values

return self.proj(g)
```

Implementation notes. We use fixed-size padding with boolean masks for nodes/edges, scatter-add for neighbor aggregation, and pool with mask awareness. All tensors are moved to CUDA when available; no MPS is used.

.7. Ablations

We summarize the ablation findings; the corresponding learning curves are in Figures 6 and 5.

- Replacing dense complexity-delta rewards with sparse outcome-only rewards (and disabling the inverse-frequency curriculum) substantially degrades performance. The main drawback of sparse rewards is *far* longer run times: we had to introduce a 1-second-per-step wall-clock timeout to keep training tractable.
- TreeMLP outperforms a vanilla GCN and GraphSAGE on the small dataset (Figure 5); the tree-aware aggregation appears necessary, not just helpful.
- In preliminary runs, adding curiosity bonuses (ICM, RND, NGU, RIDE, RE3, E3B) on top of TreeMLP did not yield consistent gains and in several cases degraded performance. This contrasts with earlier work (Dabelow & Ueda, 2024), where curiosity *improved* a vanilla MLP policy. A possible explanation is that the TreeMLP's structural inductive bias already supplies the exploration signal curiosity provides for the MLP, making the two redundant or actively conflicting. We leave a full sweep across curiosity methods $\times$ seeds for the camera-ready version.

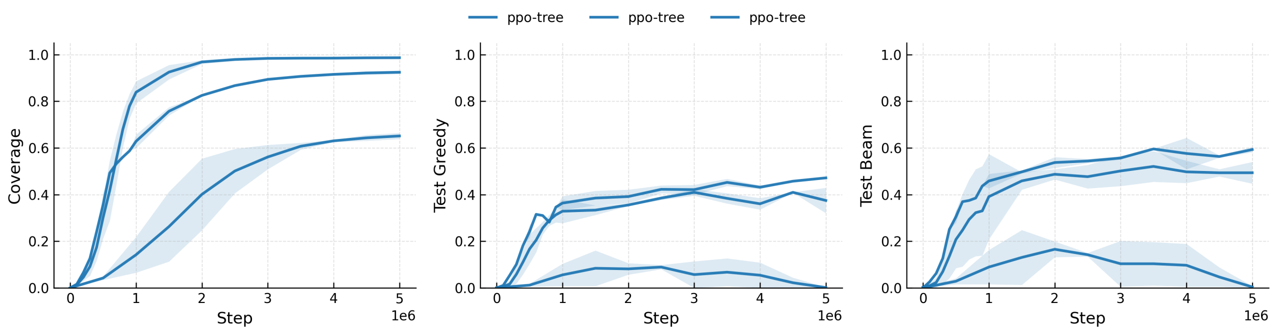

Figure 5. TreeMLP outperforms GCN and GraphSAGE on the small datasets. Mean of three random seeds with shading to min/max.

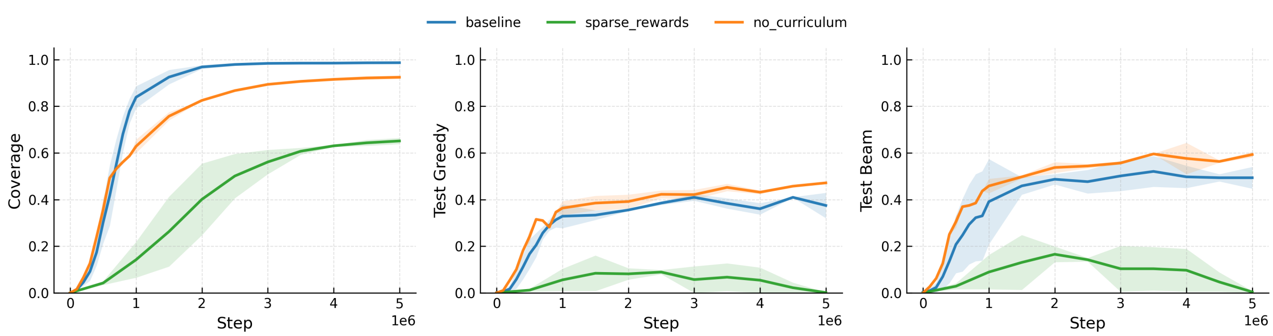

Figure 6. Ablation study on small datasets.